Forensic Readiness Logging

What to record on a Linux server *before* an incident, so that an investigation has evidence to work with · Debian/Ubuntu and RHEL-family · AWS notes where they apply

Principles. (1) **Evidence that is not recorded does not exist** - Linux keeps almost no history of processes or connections by default. (2) **Ship logs off the box immediately** - an attacker with root edits or wipes local logs; a remote, append-only copy is what you can trust. (3) **Synchronized clocks** - without them, events from different sources cannot be correlated. (4) **Keep it long enough** - intrusions are often discovered months later; plan for 90 days hot and a year or more archived. (5) **Alert when logging stops** - a silent host is a finding, not peace and quiet.

What each layer answers

#	Layer	Question it answers
1	Time and journal basics	When did it happen, in what order?
2	Central off-box logging	Can I still trust the logs after root was lost?
3	auditd	Which commands ran, who changed which critical file?
4	Sessions and shell	What did a person (or attacker) type?
5	Runtime / eBPF agents	Which process connected where, what ran in containers?
6	File integrity and baselines	What changed on disk compared with normal?
7	Network	Which IPs, domains and volumes - was data stolen?
8	Applications and databases	How did they get in, what did they access?
9	Cloud (AWS)	What happened around the box - API calls, roles, other services?
10	Readiness for memory and disk capture	Can I capture evidence fast when it happens?

1 - Time and the system journal

- ☐ Run **chrony** (or systemd-timesyncd) on every host; on AWS use the Amazon Time Sync Service (`169.254.169.123`). Check with `chronyc tracking`.
- ☐ Keep servers in **UTC** (`timedatectl set-timezone UTC`) so timestamps line up everywhere.
- ☐ Make the journal **persistent** and large enough - on some systems it lives only in RAM and vanishes on reboot.
- ☐ Set log rotation for `/var/log` (logrotate) to keep weeks locally, not days; include `wtmp`/`btmp` (login records).
- ☐ Give `/var/log` its own partition or volume so a full disk does not stop logging.

```
# /etc/systemd/journald.conf.d/10-forensics.conf
[Journal]
Storage=persistent
Compress=yes
SystemMaxUse=4G
MaxRetentionSec=90day
ForwardToSyslog=yes          # hand everything to rsyslog for remote shipping
# then: sudo systemctl restart systemd-journald
```

2 - Central, off-box, append-only logging

- ☐ Forward all logs in real time to a remote collector: rsyslog over TLS, or an agent (**Vector**, **Fluent Bit**, **CloudWatch agent**, Elastic Agent, Splunk forwarder).
- ☐ The destination must be **out of the attacker's reach**: a separate account or server, where the server's own credentials can only write, never delete. On AWS: CloudWatch Logs / S3 in a log-archive account, S3 **Object Lock** for immutability.
- ☐ Include: journal/syslog, `auth.log`/`secure`, audit log, application, web and database logs.
- ☐ Alert when a host **stops sending** logs, and on events like `auditd` being stopped or logs being truncated.
- ☐ Index them in a SIEM (Elastic, Wazuh, Splunk, OpenSearch, Sentinel) so you can search across hosts during an incident.

```
# /etc/rsyslog.d/90-remote.conf - forward everything over TLS
# (disk-backed queue keeps logs while the collector is down)
$DefaultNetstreamDriverCAFile /etc/ssl/certs/ca-certificates.crt
action(type="omfwd" target="logs.example.internal" port="6514" protocol="tcp"
       StreamDriver="gtls" StreamDriverMode="1" StreamDriverAuthMode="x509/name"
       queue.type="LinkedList" queue.filename="fwdq" queue.maxDiskSpace="1g"
       queue.saveOnShutdown="on" action.resumeRetryCount="-1")
```

3 - auditd: the kernel's own record

auditd records events in the kernel, before a user-space rootkit can hide them. It is the most important single addition to a stock Linux box. Start from a maintained ruleset (Neo23x0/auditd, the CIS rules, or the samples in `/usr/share/audit/sample-rules/`) and tune out noise.

- ☐ Install and enable: `apt install auditd audispd-plugins` / `dnf install audit`.
- ☐ Log **every executed command** (execve) with its arguments, user and parent.
- ☐ Watch identity, privilege and persistence files (below).
- ☐ Log kernel module loading, ptrace, mounts, time changes and outbound `connect` calls.
- ☐ Make rules **immutable** (`-e 2`) so an attacker cannot quietly turn auditing off without a reboot.
- ☐ Forward audit events to the central log (audisp syslog plugin, or the agent reads `/var/log/audit/`).
- ☐ Size it: execve logging is verbose on busy hosts - exclude known-noisy service accounts rather than turning it off.

```
# /etc/audit/rules.d/50-forensics.rules  (load: sudo augenrules --load)
-D
-b 8192
-f 1
# every command executed
-a always,exit -F arch=b64 -S execve -k exec
-a always,exit -F arch=b32 -S execve -k exec
# outbound network connections (can be noisy - scope with -F auid>=1000 if needed)
-a always,exit -F arch=b64 -S connect -F a2!=110 -k net_connect
# identity and privilege
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/group -p wa -k identity
-w /etc/sudoers -p wa -k priv
-w /etc/sudoers.d/ -p wa -k priv
# persistence locations
-w /etc/ssh/sshd_config -p wa -k sshd
-w /root/.ssh/ -p wa -k ssh_keys
-w /etc/cron.d/ -p wa -k cron
-w /etc/crontab -p wa -k cron
-w /var/spool/cron/ -p wa -k cron
-w /etc/systemd/system/ -p wa -k systemd
-w /etc/ld.so.preload -p wa -k preload
-w /etc/pam.d/ -p wa -k pam
# kernel modules, ptrace, time
-a always,exit -F arch=b64 -S init_module,finit_module,delete_module -k modules
-a always,exit -F arch=b64 -S ptrace -k ptrace
-a always,exit -F arch=b64 -S adjtimex,settimeofday,clock_settime -k time
# lock the configuration (needs reboot to change)
-e 2
```

Search later with `ausearch -k exec -i` and `aureport -x --summary`.

4 - Sessions, sudo and shell history

- ☐ **sudo I/O logging** records full keystrokes and output of every sudo session; replay with `sudoreplay`.
- ☐ Record whole SSH sessions with **tlog** (RHEL) or by routing admin access through **SSM Session Manager** with logging to S3/CloudWatch.
- ☐ Timestamp shell history and write it immediately (below). Treat it as a convenience - attackers unset it; auditd execve is the reliable record.
- ☐ Enable **process accounting** (`acct` / `psacct`): cheap record of every process, queried with `lastcomm` and `sa`.
- ☐ Log successful and failed logins (default) and keep `wtmp`/`btmp` for months.

```
# visudo -f /etc/sudoers.d/10-logging
Defaults log_input, log_output
Defaults iolog_dir=/var/log/sudo-io/%{user}
Defaults logfile=/var/log/sudo.log

# /etc/profile.d/history.sh
export HISTTIMEFORMAT='%F %T '
export HISTSIZE=100000 HISTFILESIZE=100000
shopt -s histappend
PROMPT_COMMAND='history -a; logger -p local6.info -t shell \
  "$(whoami) [$$]: $(history 1 | sed "s/^ *[0-9]* *//")"'
readonly HISTTIMEFORMAT HISTSIZE HISTFILESIZE PROMPT_COMMAND
```

5 - Runtime and eBPF security agents

These see process trees, network connections and file access with full context - which process, which container, which parent - and are far richer than auditd. Pick one or two:

Tool	Strength
Falco	Real-time rules for suspicious behavior (shell in container, reverse shell, writes to /etc); CNCF project
Tetragon	eBPF process execution and network events with full ancestry; can also block
osquery	SQL over the system: scheduled snapshots of processes, listening ports, users, crontabs, authorized_keys, kernel modules - shipped off-box as a history
Sysmon for Linux	Microsoft Sysmon events (process create, network connect, file create) via eBPF
Wazuh	Open-source agent + SIEM: log collection, FIM, rootkit checks, alerting in one package
GuardDuty Runtime Monitoring	AWS-managed eBPF agent for EC2/EKS/ECS, findings in GuardDuty
Commercial EDR	CrowdStrike, SentinelOne, Defender for Endpoint, Elastic Defend - recording plus response

6 - File integrity and baselines

- ☐ **AIDE** (or Wazuh/osquery FIM) over `/etc`, `/bin`, `/sbin`, `/usr`, `/lib`, `/boot` and web roots; daily check; database and reports stored off the box.
- ☐ Keep a **baseline** of the known-good state, refreshed after every change: installed packages with versions, listening ports, enabled services and timers, users and groups, SUID files, sudo rules, kernel modules, authorized_keys. Diffing against it is the fastest way to find what an attacker added.
- ☐ Record the **golden image** (AMI ID, image build) each server came from.
- ☐ Take **scheduled EBS snapshots** (AWS Data Lifecycle Manager / AWS Backup) - a pre-incident disk to compare with is extremely valuable.
- ☐ Keep application code in git and deploy from it, so changes on disk (e.g. web shells) show up as a diff.

```
# baseline snapshot - run after changes, store the output off the box
B=/var/tmp/baseline-$(date -u +%F); mkdir -p $B
dpkg -l > $B/packages.txt 2>/dev/null || rpm -qa | sort > $B/packages.txt
ss -tulpn > $B/listening.txt;  systemctl list-unit-files --state=enabled > $B/services.txt
systemctl list-timers --all > $B/timers.txt;  lsmod > $B/modules.txt
getent passwd > $B/passwd.txt;  getent group > $B/group.txt
find / -xdev -perm /6000 -type f 2>/dev/null | sort > $B/suid.txt
cat /etc/sudoers /etc/sudoers.d/* > $B/sudoers.txt 2>/dev/null
for h in /root /home/*; do
  echo "== $h"; cat $h/.ssh/authorized_keys 2>/dev/null
done > $B/authorized_keys.txt
```

7 - Network records

- ☐ **VPC Flow Logs** to S3 with the extended fields (instance-id, tcp-flags, pkt-srcaddr/dstaddr, flow-direction).
- ☐ **DNS query logging**: Route 53 Resolver query logs on AWS, or a local resolver that logs queries (unbound `log-queries: yes`, dnsmasq `log-queries`). DNS reveals C2 domains and tunneling.
- ☐ Log **new outbound connections** on the host with nftables (below), or conntrack events (`conntrack -E` via `ulogd2`/`conntrackd`).
- ☐ For high-value servers: a **Zeek** or **Suricata** sensor fed by AWS Traffic Mirroring or a SPAN port - per-connection, DNS, HTTP and TLS logs (with JA3/JA4) without storing all packets.
- ☐ Optional **rolling packet capture** (e.g. `tcpdump -i eth0 -G 3600 -W 48 -w cap-%F-%H.pcap`) for the last 48 hours - expensive in disk, decisive in an investigation.

- ☐ TLS key logging (eCapture / SSLKEYLOGFILE) only with strong protection of the key log - it can decrypt every captured session.

```
# add to the nftables output chain - log the first packet of every new outbound connection
# group 0 -> ulogd2; drop "group 0" to log to the kernel log instead
ct state new log prefix "OUT-NEW " group 0
```

8 - Applications and databases

- ☐ Web server access logs with: client IP (and `X-Forwarded-For` behind a proxy), timestamp with timezone, method, full URL, status, bytes, response time, user agent, referrer, virtual host.
- ☐ Load balancer, CDN and WAF logs - they hold the real client IP and requests the server never saw.
- ☐ Application logs for **security events**: logins (success and failure), password resets, permission changes, admin actions, file uploads, API key use - with user ID and source IP.
- ☐ Database audit logging: **pgaudit** (PostgreSQL), MariaDB/MySQL audit plugin; at least logins, DDL and privilege changes, and access to sensitive tables.
- ☐ Never log passwords, tokens or full card numbers - logs are evidence, and also a target.

9 - Cloud records (AWS)

- ☐ **CloudTrail**: an organization trail in all regions, log file validation on, delivered to a separate log-archive account; add **data events** for sensitive S3 buckets and Lambda.
- ☐ **GuardDuty** in all regions (including Runtime Monitoring and S3 protection); export findings to S3.
- ☐ **AWS Config** for configuration history of security groups, IAM, instances.
- ☐ **SSM Session Manager** logging to S3/CloudWatch; S3 server access logs; ELB/ALB access logs.
- ☐ Retention policies on CloudWatch log groups (the default is *never expire* - set it deliberately, and archive to S3 Glacier for the long term).

10 - Readiness for memory and disk capture

- ☐ Pre-stage **AVML** (static binary) and, if you prefer, **LiME** built for each kernel you run.
- ☐ Generate **Volatility 3 symbol tables** (`dwarf2json` from the kernel's debug symbols) for each deployed kernel version - without them, a memory image is very hard to analyze.
- ☐ Have an evidence location ready: an S3 bucket with Object Lock, or an attachable evidence volume.
- ☐ Keep the incident-response and hardening checklists printed and stored off the servers.
- ☐ **Test it:** once a quarter, simulate an intrusion on a test box (add a user, a cron job, a reverse shell to a test IP) and check that you can reconstruct it end to end from the central logs alone.

References: NIST SP 800-92 (Log Management), NIST SP 800-86 (Forensic Techniques), CIS Benchmarks (logging and auditing sections), AWS Security Incident Response Guide.